CitationGuard: Checking Scientific Claims Against Their Evidence

Phase 3 Report: Model Building

Course: Artificial Intelligence · Group ID: **F26-04**
Muhammad Asjad (24L-2570, Team Lead) · Hamza Sheikh (24L-2500) · Haider Ali (24L-2522)
Code: F26-04.ipynb and src/phase3_pipeline.py · GitHub: https://github.com/Asjadamir/CitationGuard

CitationGuard checks whether a scientific claim is really backed up by the paper it cites. In Phase 2 we cleaned the SciFact dataset and explored it. In Phase 3 we built and ran our first two models: a **finder** that searches for evidence sentences, and a **checker** that decides whether the evidence supports the claim. In the code and charts, the finder is called *retrieval* (Phase 3A) and the checker is called *verifier* (Phase 3B).

1. Model selection and why

Finder: TF-IDF + cosine similarity. TF-IDF turns each sentence into a list of numbers based on its words. Words that are rare in the corpus get more weight. Cosine similarity then measures how close the claim is to each sentence. For every claim, the finder ranks all 45,952 abstract sentences in SciFact.

- SciFact has 5,183 papers but only about 1,400 labeled claims. That is too few labels to train a neural search model, while TF-IDF needs no labels at all.
- Claims and their evidence often share exact technical words (gene, drug and disease names), and TF-IDF is good at matching those words.
- It is fast, runs on a normal laptop, and is a standard first baseline. In Phase 2 we planned to test TF-IDF first, before BM25 or dense retrieval.

Checker: TF-IDF + Logistic Regression. The checker reads a claim together with an evidence text and gives one of three labels: SUPPORT, CONTRADICT or NO_EVIDENCE. Logistic Regression learns a weight for each word group and each label, and picks the label with the highest score.

- It is a standard baseline for text classification. It trains in seconds and needs no GPU.
- It is simple enough for our team to understand and explain step by step, and it gives a clear starting point to compare later models against.
- Neural models (for example NLI or cross-encoder models) should be better at SUPPORT vs CONTRADICT. We left them for later, because the goal of Phase 3 was a correct, working and repeatable pipeline.

2. Implementation details

Tool	What we used it for
Python 3	Main programming language.
scikit-learn	`TfidfVectorizer`, `LogisticRegression`, `Pipeline`, `cosine_similarity`, and the metrics (accuracy, F1, classification report, confusion matrix).
pandas	Loading the Phase 2 CSV files and saving the predictions.
NumPy	Sorting sentences by score and averaging the metrics.
matplotlib	The two Phase 3 charts (Figures 1 and 2).
Jupyter Notebook	F26-04.ipynb runs both models and shows all outputs.

Everything runs on a normal laptop CPU. No GPU is needed.

3. Pipeline: data in, model, output

	Data in	Model	Output
Finder	Raw SciFact files: corpus.jsonl (5,183 abstracts, 45,952 sentences) and claims_train/dev.jsonl.	Each "title. sentence" becomes a TF-IDF vector, and so does the claim. Cosine similarity scores every sentence.	Ranked sentences for each claim (top 5 shown). Scores: Recall@k, Hit@k, MRR.
Checker	Phase 2 CSVs: verification_train/dev.csv (SUPPORT, CONTRADICT) and no_evidence_train/dev.csv (NO_EVIDENCE).	claim + " [SEP] " + evidence text → TF-IDF vector → Logistic Regression.	A label and a probability for each label, per row. Scores: accuracy, F1, confusion matrix.

All outputs (metrics, predictions and charts) are saved in results/ and figures/.

- **The finder and checker run separately for now.** The checker gets the correct evidence from the dataset, not the sentences the finder found. So the checker's scores show how well it labels a claim when the evidence is already known. Joining the two into one system is future work.
- SUPPORT and CONTRADICT rows hold the gold evidence sentences that SciFact's annotators marked in a cited abstract. NO_EVIDENCE rows hold the whole abstract of a paper that the claim cites but that has no evidence for it.
- The finder reads the raw SciFact files, not our Phase 2 corpus_clean.csv. The reason is that corpus_clean.csv stores each abstract as one long text, but the finder needs each abstract split into sentences, plus the numbers of the gold sentences. The notebook downloads the raw files into data/raw/.
- "[SEP]" is only a separator between the claim and the evidence. For TF-IDF it is just one more word ("sep") that appears in every row, so it has almost no effect.
- The model code lives in `src/phase3_pipeline.py` in our GitHub repo. The notebook imports `run_retrieval()` and `run_verifier()` from it, so it must be run from the repo root. Repo: https://github.com/Asjadamir/CitationGuard

4. Training process

We use the official SciFact train/dev split with no random reshuffle, and no claim appears in both sets. SciFact hides the answers for its test set, so we use dev as our test set and never train on it.

Split	SUPPORT	CONTRADICT	NO_EVIDENCE	Checker rows	Finder claims*
Train	616	341	355	1,312	505 of 809
Dev (our test set)	216	122	130	468	188 of 300

* The finder is scored only on claims that have gold evidence sentences.

Setting	Finder	Checker
Word groups	1-2 words (`ngram_range=(1, 2)`)	1-2 words
min_df	2	2
max_features	50,000	40,000
sublinear_tf	True	True
Stop words	English stop words removed, lowercase	English stop words removed, lowercase
Scoring / classifier	Cosine similarity	Logistic Regression: `solver=lbfgs`, `C=1.0`, `max_iter=4000`, `random_state=42`
Fitted on	Corpus sentences only (no claims, no labels)	1,312 train rows
Tested on	505 train claims and 188 dev claims	468 dev rows

Epochs: the models do not use epochs (an epoch is one full pass over the data when training a neural network). TF-IDF is computed in one pass. Logistic Regression was allowed up to 4000 steps (`max_iter=4000`) and stopped at 25, because it had already converged.

5. Initial results

Finder (dev, 188 claims, searching all 45,952 sentences). Recall@k is the share of gold evidence sentences found in the top k. Hit@k is the share of claims with at least one gold sentence in the top k. MRR is the average of 1 / (rank of the first gold sentence), so 1.0 would mean the first result is always right.

	k = 1	k = 3	k = 5	k = 10
Recall@k	0.240	0.385	**0.473**	0.554
Hit@k	0.335	0.500	**0.601**	0.670

MRR on dev is **0.447**. So for 60% of dev claims, a correct evidence sentence is in the top 5 out of almost 46,000 sentences. Train claims give similar scores (MRR 0.422, Hit@5 0.592). Example: for the claim "1,000 genomes project enables mapping of genetic sequence variation consisting of rare variants with larger penetrance effects than common variants", the gold sentence came 3rd; the top 2 were related sentences about rare variants from other papers.

Checker (dev, 468 rows). Accuracy is the share of rows labeled correctly. Macro F1 is the average F1 of the three labels, with each label counting the same. The baseline always answers SUPPORT, the most common label.

Accuracy	Macro F1	Weighted F1	Baseline "always SUPPORT"
0.551	**0.522**	0.535	0.462

Label	Precision	Recall	F1	Dev rows
SUPPORT	0.516	0.607	0.557	216
CONTRADICT	0.126	0.090	0.105	122
NO_EVIDENCE	0.913	0.892	0.903	130

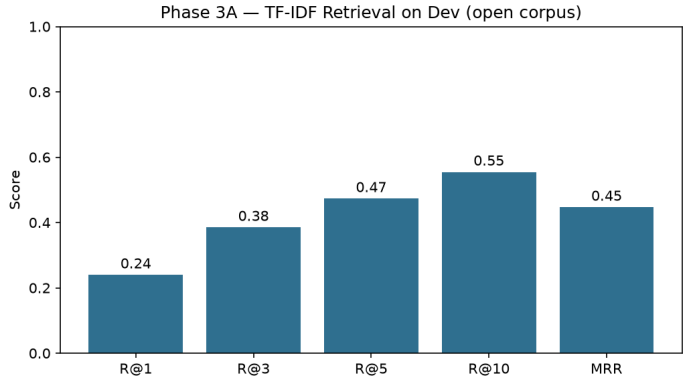


Figure 1: Finder scores on dev (188 claims).

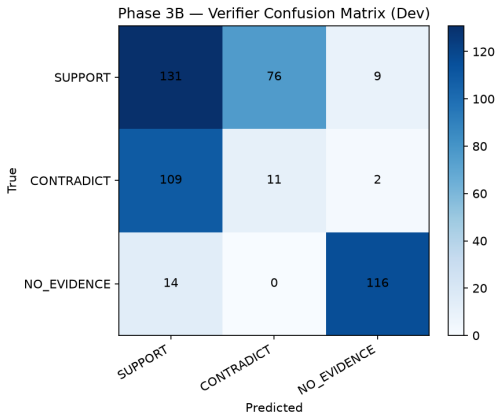


Figure 2: Checker confusion matrix on dev (468 rows).

Both models run from start to end and produce real outputs. The checker beats the baseline, but the scores per label are very uneven. Section 7 explains why.

6. Is this a RAG project? No.

RAG (retrieval-augmented generation) means finding relevant text and then giving it to a large language model (a chat-style model) that writes an answer, guided by a system prompt. Our project does search for text, but it has no text-generating model and no system prompt. The finder uses TF-IDF word-count vectors, not neural embeddings, and the checker is a classifier that picks one of three labels. So the RAG-only requirement (embedding pipeline and system prompts) does not apply to our project.

7. Challenges

- **Text-length shortcut.** NO_EVIDENCE rows contain a whole abstract (about 155 words), while SUPPORT and CONTRADICT rows contain only 1-2 evidence sentences (about 29 words). So the model may be using text length as a shortcut to spot NO_EVIDENCE, and its high NO_EVIDENCE score (F1 0.903) does not prove that it understands the text. Giving all three labels inputs of similar length would make the test fairer.
- **SUPPORT vs CONTRADICT barely works.** Only 11 of 122 CONTRADICT rows are right, and 109 are predicted as SUPPORT. On these two labels alone the model is right 142 of 338 times (42%), while always saying SUPPORT would be right 216 times (64%). So the win over the baseline comes mainly from NO_EVIDENCE. TF-IDF only sees which words appear, not what they mean together. For example, “ALDH1 expression is associated with better breast cancer outcomes” with the evidence “…correlated with poor prognosis” was labeled SUPPORT, because the two texts share most of their words.
- **Class imbalance.** The train set has 616 SUPPORT rows but only 341 CONTRADICT rows, which pushes the model towards SUPPORT.
- **Searching a large open corpus.** The finder must find about 2 gold sentences among 45,952. When the claim and the evidence use different words for the same idea, TF-IDF misses the match, as in the example in Section 5.
- **Small differences between computers.** The finder keeps the 50,000 most common word groups, but many word groups tie for the last places, and different NumPy versions break these ties differently. Our numbers come from NumPy 2; with NumPy 1.26 the dev MRR was 0.456 instead of 0.447. The checker gave exactly the same numbers in both setups.
- **Next steps.** Try BM25 and dense retrieval for the finder, try a neural NLI or cross-encoder model for the checker, and connect the two so that the checker uses the finder's sentences.

8. Credit

- **Dataset:** D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan and H. Hajishirzi. “Fact or Fiction: Verifying Scientific Claims.” EMNLP 2020. Data: github.com/allenai/scifact
- **scikit-learn:** F. Pedregosa et al. “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research 12, pp. 2825-2830, 2011.
- We also used the open-source Python libraries pandas, NumPy and matplotlib.